- Consider the update to $sleep(E)$ and $wut(E)$ at lines 20 and 21. We must prove that they preserve the two properties of Definition 6.1.
  (a) Since the update removes nodes from $wut(E)$, and p is added to $sleep(E)$, it suffices to check that $p \notin WI_{[E]}(w)$ whenever w is a leaf of $wut(E)$ that remains after the operation, i.e., w is not of form $p.w'$. Before the operation, it was the case that p and w were nodes of $wut(E)$ with $p \prec w$ and w a leaf. From the inductive hypothesis, Property 2 of Definition 6.1 implies that $p \notin WI_{[E]}(w)$, which completes the proof for this case.
  (b) Since $wut(E)$ is modified by removing a branch from the root, this property is preserved.

(2) Consider a new call to $Explore(E.p, Sleep', WuT')$ at line 19. If WuT' is empty, then p must have been a leaf in $wut(E)$ just before the call. By Property 1 of the inductive hypothesis, $wut(E)$ was then a wakeup tree after $\langle E, sleep(E)\rangle$, therefore (by Property 1 of Definition 6.1), we have $E \nvdash p \diamond q$ for all $q \in sleep(E)$, which by construction of $Sleep'$ at line 17 implies that $Sleep'$ is empty.

□

Lemma 7.2. *If $E.p \propto E.w$ then $p \notin I_{[E]}(w)$.*

Proof. We use a proof by contradiction. Assume that $E.p \propto E.w$ but $p \in I_{[E]}(w)$. The property $p \in I_{[E]}(w)$ implies that w is of form $w'.p.w''$ with $E \vdash p \diamond w'$. The property $E.p \propto E.w$ implies that the sequence $E.w$ is explored after the call to $Explore(E.p, \cdot, \cdot)$ has returned. After the call to $Explore(E.p, \cdot, \cdot)$ has returned, we have that $p \in sleep(E)$, by the insertion at line 21. By the way sleep sets are updated at line 17, and the assumption $E \vdash p \diamond w'$, we conclude that p remains in the sleep set at the calls $Explore(E.w''', \cdot, \cdot)$ whenever w''' is a prefix of w'. Thus $p \in sleep(E.w')$ just before the call $Explore(E.w'.p, \cdot, \cdot)$ is performed. By Invariant 1 of Lemma 7.1, we then have that $wut(E.w')$ is a wakeup tree after $\langle E.w', sleep(E.w')\rangle$. To perform the call $Explore(E.w'.p, \cdot, \cdot)$, it must be the case that p is a node in $wut(E.w')$. But this contradicts Property (1) of Definition 6.1, which says that $p \notin WI_{[E.w']}(p.v)$ where v is chosen so that $p.v$ is a leaf of $wut(E.w')$. Thus, the lemma is proven. □

The following lemma captures the relationship between wakeup trees and $\langle \mathcal{E}, \propto\rangle$.

Lemma 7.3. *Let $\langle \mathcal{E}, \propto\rangle$ be the tree of explored execution sequences. Consider some point in the execution, and the wakeup tree $wut(E)$ at that point, for some $E \in \mathcal{E}$.*

(1) If $w \in wut(E)$ for some w, then $E.w \in \mathcal{E}$.
(2) If $w \prec w'$ for $w, w' \in wut(E)$ then $E.w \propto E.w'$

Proof. The properties follow by noting how the exploration from any $E \in \mathcal{E}$ is controlled by the wakeup tree $wut(E)$ at lines 15–21. □

We can now prove that Algorithm 2 is correct in the sense that for each maximal execution sequence E, it explores an execution sequence in $[E]_{\simeq}$. This is formalized in Theorem 7.4. Similarly to Lemma 5.3, Theorem 7.4 is proven by induction on the order in which states (i.e., execution sequences) are backtracked by the algorithm. The inductive step is proven by contradition, assuming that some maximal sequence $E.w$ is unexplored after the call for E returns. The proof of this inductive step is structured as a sequence of claims, structured in the way shown in Fig. 8. First it is shown that the assumption implies Claim 7.5, which is analogous to Claim 5.4 in the proof of Lemma 5.3, and essentially states that w is a counterexample to the statement that $final_sleep(E)$ is a source set for the set of sequences w with $E \vdash w$. Thereafter, the sequence of Claims 7.6–7.9 establish that the